\WOOLF/

Applied Software Project Report

NutriFlow: Intelligent Calorie and Macro Tracking System

By

Soubhik Mazumdar

A Master's Project Report submitted to Scaler Neovarsity - Woolf

in partial fulfillment of the requirements for the degree of

Master of Science in Computer Science

Month of Submission: April 2026

Scaler Mentee Email ID: mazumdarsoubhik@gmail.com

Thesis Supervisor: Naman Bhalla

Date of Submission: 26/04/2026

Certification

I confirm that I have overseen and reviewed this applied project and, in my judgment, it adheres to the appropriate standards of academic presentation. I believe it satisfactorily meets the criteria, in terms of both quality and breadth, to serve as an applied project report for the attainment of the Master of Science in Computer Science degree. This applied project report has been submitted to Woolf and is deemed sufficient to fulfill the prerequisites for the Master of Science in Computer Science degree.

Naman Bhalla

Project Guide / Supervisor

Declaration

I confirm that this project report, submitted to fulfill the requirements for the Master of Science in Computer Science degree, completed by me from January 2026 to April 2026, is the result of my own individual endeavor. The project has been made on my own under the guidance of my supervisor with proper acknowledgement and without plagiarism. Any contributions from external sources or individuals, including the use of AI tools, are appropriately acknowledged through citation. By making this declaration, I acknowledge that any violation of this statement constitutes academic misconduct. I understand that such misconduct may lead to expulsion from the program and/or disqualification from receiving the degree.

Project Team

Candidate

Date: 25 April 2026

Acknowledgment

I express my sincere gratitude to my family for their constant encouragement and support throughout this journey. I thank the Scaler instructors, mentors, and peers for their guidance, technical feedback, and motivation during the backend specialization program. I am especially grateful to my project supervisor, Naman Bhalla, for valuable direction and review support. Their collective help and inspiration made this project possible and enabled successful completion of the Master's degree requirements.

Table of Contents

Contents

List of Tables

List of Figures

NutriFlow is a low-friction nutrition tracking system built to address a common product problem: users start calorie tracking but quickly abandon it because meal logging is slow and mentally tiring. The project focuses on fast, natural-language capture with useful daily feedback. Instead of rigid form entry, users can submit meal text such as "2 roti, dal, curd," and the system converts it into structured items with estimated calories, protein, carbohydrates, fat, and fibre. The goal is practical consistency, not clinical precision.

The implemented solution follows a modular loop with three surfaces: meal logging, daily dashboard, and nutrition chat. A FastAPI backend exposes versioned APIs for auth, meal CRUD, history, dashboard aggregation, and chat. A Streamlit frontend provides Today, History, Chat, and Settings views. Data is managed with SQLAlchemy models for users, goals, foods, meals, meal items, daily summaries, and chat messages. SQLite is used for local development with a migration path to PostgreSQL.

The parser combines deterministic normalization and alias-based food resolution, with explicit fallback assumptions for unknown foods or missing quantities. This keeps logging resilient under imperfect input. After each meal mutation, daily summaries are recomputed to provide consumed/target/remaining macro values and streak indicators. Chat responses are concise and context-aware, based on day-level nutrition gaps. By deliberately excluding higher-scope features in this phase, NutriFlow demonstrates an end-to-end backend-centered system that improves adherence through speed, clarity, and repeatable daily workflow.

I. Project Description

NutriFlow was conceived as a response to a practical product gap in consumer health applications. Many users understand the value of tracking calorie and macronutrient intake, but most do not maintain the habit for more than a short period. Existing applications often demand high-precision input, extensive search interactions, and too many UI steps per meal. In real-world conditions, especially for working professionals and beginners, these requirements create friction that is stronger than motivation. NutriFlow reframes the problem from "how to maximize nutritional precision" to "how to maximize daily logging consistency while retaining useful guidance." This shift defines the product, the technical architecture, and the implementation priorities documented in this report.

The central objective of NutriFlow is to make meal logging possible in under a few seconds per entry while still providing meaningful visibility into calories and macros. The system encourages users to log quickly in plain language, then translates those entries into structured nutrition data that can be aggregated over the day. Instead of presenting tracking as a compliance-heavy data entry burden, NutriFlow positions tracking as a lightweight reflection loop: log food, view updated progress, understand remaining targets, and decide the next meal more consciously. This loop is

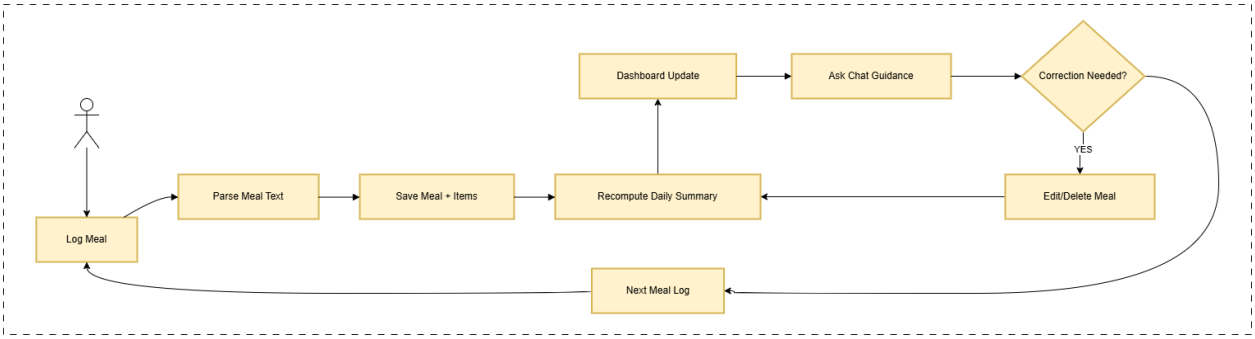

Fig. 1 Project flow and user journey showing the loop from meal logging to parsing, dashboard feedback, chat guidance, and correction actions.

intentionally narrow and repeatable. Features that do not strengthen this loop are deprioritized in the current scope.

The intended users are individuals who want practical nutrition awareness but are not willing to perform detailed food journaling every day. Primary personas include office-going professionals, students, and early-stage fitness users who need direction but not complexity. For these users, the product must satisfy three requirements simultaneously: low effort of use, immediate interpretability of output, and flexibility for imperfect input. NutriFlow addresses each requirement through explicit design choices. It accepts free-form text instead of requiring rigid forms, exposes consumed/target/remaining values to reduce mental calculations, and allows edits/deletes in history so users can correct approximate entries without losing trust in the system.

The project scope is structured around three product surfaces. The first surface is meal logging, which acts as the primary entry point and highest-frequency interaction. In the implemented backend, `POST /api/v1/meals` receives text, applies parsing logic, computes nutrition totals, stores normalized meal items, and returns a typed response with confidence and totals. The second surface is dashboard feedback, exposed through `GET /api/v1/dashboard/today`, where daily macro consumption is compared against user-specific targets and converted into actionable "remaining" values. The third surface is contextual guidance through `POST /api/v1/chat`, where simple assistant responses are generated from day-level macro gaps and user prompts. Together, these surfaces operationalize the product promise: fast input, visible progress, and lightweight guidance.

From a software architecture perspective, NutriFlow is implemented as a modular monolith in FastAPI. This structure was chosen because it offers deployment simplicity while keeping boundaries clear for future decomposition. API modules are separated by concern (`meals`, `dashboard`, `chat`, `health`). Service modules encapsulate business logic (`parser`, `meals`, `dashboard`, `food_reference`, `users`, `chat`). Data persistence is modeled through SQLAlchemy entities with explicit relationships and cascade behavior where needed. The core tables in the current implementation are `users`, `user_goals`, `foods`, `meals`, `meal_items`, `daily_summaries`, and `chat_messages`. This schema supports both transactional meal workflows and day-level analytical aggregation without introducing unnecessary infrastructure complexity at this stage.

The parsing strategy reflects a deliberate trade-off between robustness and operational cost. Deterministic parsing is used first: text is normalized, split into chunks, quantity tokens are interpreted (numeric and word-based), and alias mappings resolve candidate foods. When quantity is missing, defaults are applied from food metadata; when a food is unknown, fallback assumptions generate editable records with lowered confidence. This approach ensures that user input is rarely rejected, which is critical for habit continuity. Confidence values and assumptions are persisted with each meal item so that uncertainty is not hidden. In practical product terms, the parser prefers "quickly log with transparent approximation" over "fail fast until input is perfect."

The dashboard subsystem is designed as the reinforcement engine of the product. After every meal create, update, or delete operation, daily summaries are recomputed for affected dates. Aggregation computes meal count and macro totals, and then combines those values with user goals to produce macro pairs (`consumed`, `target`, `remaining`). A streak calculation runs backward over daily summaries to quantify consecutive logging days, adding a behavioral signal that supports retention. The frontend renders these outputs as concise metrics rather than dense analytics, keeping the interface legible for users who need immediate interpretation during daily routines.

The chat subsystem is intentionally constrained. It is not positioned as a medical or therapeutic advisor; it provides concise, context-aware suggestions based on logged intake and user prompts. For example, when protein or fibre gaps are present, responses include practical meal suggestions with clear next actions. This design avoids overpromising model intelligence while still adding value at decision points. In both BRD and implementation terms, AI serves as a support layer around the core logging/dashboard loop, not as the center of the product.

The frontend implementation in Streamlit reflects a rapid-delivery strategy focused on validating workflows end to end. The Today tab supports logging and same-day review. The History tab supports date-filtered retrieval, pagination, edit, and delete actions. The Chat tab maintains conversational context and integrates backend responses. The Settings tab allows backend URL configuration and logout handling while authenticated requests use bearer access tokens. This frontend is intentionally pragmatic: it prioritizes functional coverage of backend capabilities and fast iteration over visual complexity.

Non-functional expectations are embedded into technical choices. The API design is synchronous and straightforward, minimizing request overhead for the primary user path. Input validation, typed schemas, and error handling provide predictable client behavior. Dependency injection for database sessions and user identity simplifies endpoint contracts. Seed food references and alias maps reduce time-to-first-log in local environments. The architecture remains deployment-flexible: while local development currently uses SQLite, models and service boundaries are aligned with migration to PostgreSQL and managed infrastructure when scaling requirements increase.

A significant contribution of this project is the alignment between business requirements and backend implementation decisions. The BRD defines low-friction logging, understandable progress, and contextual guidance as core outcomes. The SDD translates these outcomes into modules, data models, and endpoint contracts. The implemented code

demonstrates this translation concretely: every major endpoint maps to a user-visible need, every major table maps to a workflow or metric, and every major service function maps to an explicit product behavior. This requirement-to-code traceability is central to the value of the project report.

The project also acknowledges current limitations. Food coverage is finite and regionally variable; macro estimates are approximations, especially for mixed home-cooked dishes; and recommendation logic in chat is intentionally simple. However, these limits are accepted within scope because they preserve the primary objective of fast, consistent use. The system already includes correction paths through meal editing and history management, which is essential for trust in approximate systems. Future enhancements can progressively improve precision and personalization through larger food corpora, selective model-assisted parsing, and more adaptive recommendation modules without disrupting existing flows.

In summary, NutriFlow is a backend-driven applied software project that demonstrates how disciplined scope and practical engineering can improve adherence in a common but high-drop-off health behavior. It does not attempt to solve every nutrition problem at once. Instead, it solves one critical product challenge well: converting meal tracking from a high-friction task into a repeatable daily workflow supported by clear feedback and lightweight intelligence. This focus is what makes the current implementation technically coherent, user-relevant, and extensible for subsequent development phases.

II. Requirement Gathering

Requirement gathering for NutriFlow was carried out with a product-first, backend-grounded approach. The objective was to define requirements that are both meaningful for end users and directly traceable to implementable API, data, and service behavior. Instead of collecting a broad set of speculative features, the process emphasized high-frequency actions that users perform daily and that determine retention outcomes. The project deliberately treated meal logging as a behavioral workflow problem rather than only a data accuracy problem. Therefore, requirement quality was evaluated on two axes at the same time: technical feasibility and expected impact on habit continuity.

The primary input sources for requirement definition were the business problem statement, user personas, implementation constraints, and iterative validation against the running system architecture. The business problem identified the key failure mode in existing nutrition products: users abandon tracking because friction per meal is too high. User personas highlighted the need for speed and low cognitive load, especially for working professionals and beginner users. Technical constraints included limited initial scope, a backend-centered specialization requirement, and the need to ship an end-to-end flow with clear API contracts. As a result, requirements were prioritized only if they supported the core product loop: log quickly, see progress immediately, and receive simple next-step guidance.

In this project, requirement gathering also included explicit exclusion decisions. Many capabilities are common in modern health applications, but not all of them improve early-stage user consistency. Features such as image recognition,

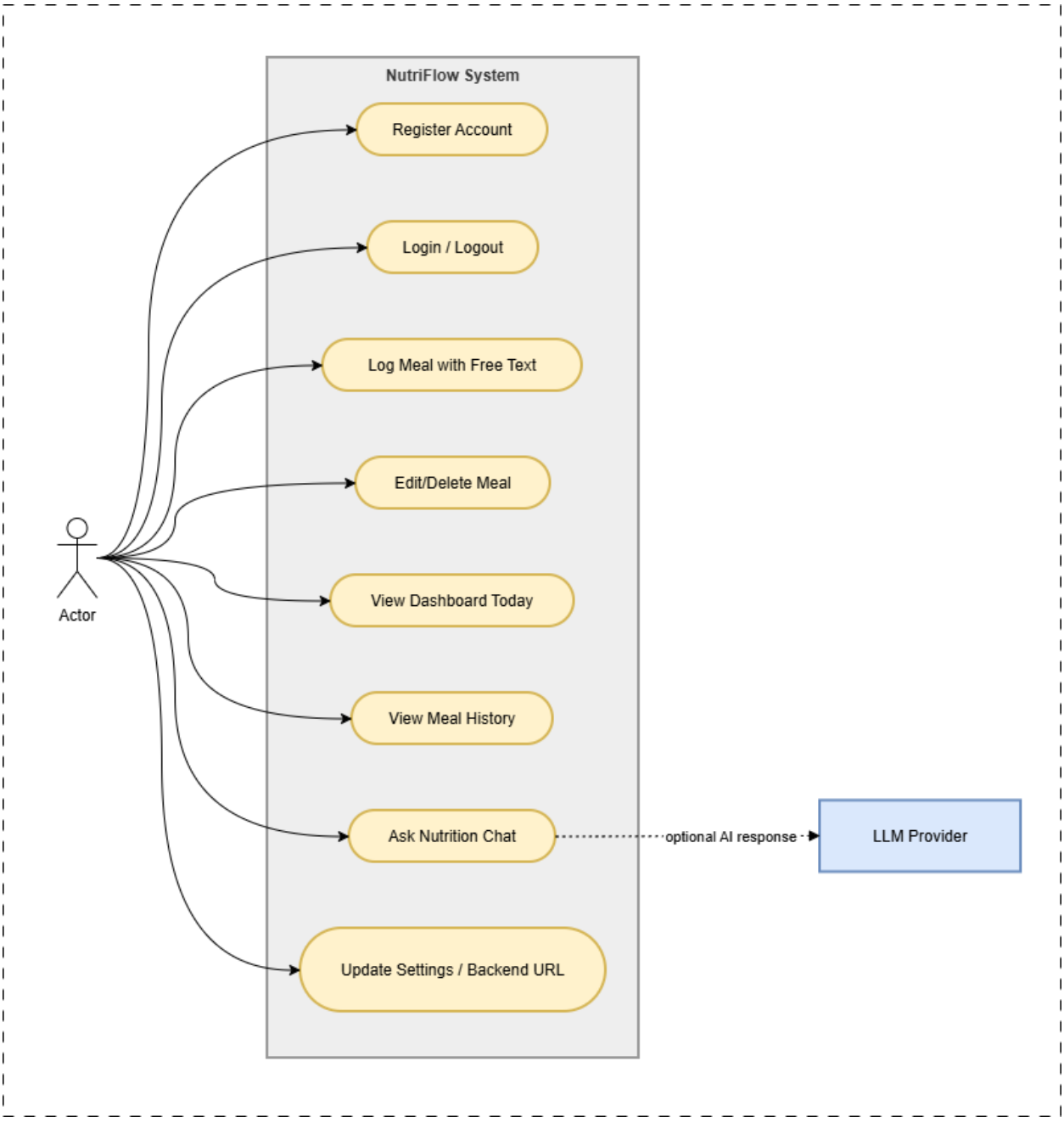

Fig. 2 Use case diagram showing primary actors and major system interactions across authentication, meal tracking, dashboard, history, chat, and settings.

barcode scanning, social competition, and clinical-grade recommendations were kept out of scope to preserve delivery speed and architectural clarity. This was not a limitation of imagination but a deliberate product strategy: complete and reliable execution of a narrow loop is preferable to partial execution of a broad feature set. The requirements below reflect this principle by balancing user value, engineering complexity, and measurable behavior outcomes.

The section is structured in three layers. First, functional requirements define what the system must do from a user and API perspective. Second, non-functional requirements define how the system should behave under performance, reliability, maintainability, and usability expectations. Third, requirement-level assumptions and acceptance criteria clarify interpretation boundaries to reduce ambiguity during design and implementation.

A. Functional Requirements

FR-01 Meal Logging Through Natural Language: The system shall accept free-text meal input from the user and process it without requiring structured manual entry per food item. The user may optionally provide contextual fields such as `meal_type` and `eaten_at`, but the core action must remain valid with only text input. The system shall parse the input into one or more item-level entries, estimate quantity and unit where possible, compute calories, protein, carbohydrates, fat, and fibre, and persist both original input and normalized records. This requirement maps to the backend flow implemented through `POST /api/v1/meals`, parsing logic in the meal parser service, and item persistence in the `meals` and `meal_items` tables. The user-facing intent is that logging should remain fast and resilient even when text is imperfect.

FR-02 Quantity and Food Interpretation with Transparent Assumptions: The system shall support mixed input patterns such as numeric quantity ("2 roti"), textual quantity ("one bowl dal"), and unspecified quantity ("curd"). For missing or ambiguous values, the system shall apply deterministic defaults from reference food metadata and store assumptions used for estimation. For unknown foods, the system shall create a fallback item with reduced confidence instead of rejecting the entire meal by default. This requirement is critical for continuity because hard rejection behavior increases abandonment. The implementation expectation is not perfect parsing but reliable conversion with traceable confidence and assumptions, enabling later correction.

FR-03 Reference Food Mapping and Macro Computation: The system shall maintain a canonical food reference with aliases so that common user terms resolve to known foods. Macro values shall be computed from canonical per-serving nutrition multiplied by inferred quantity factors. Alias handling shall support regional and colloquial naming where available. This requirement enables consistency of outputs across repeated logs and reduces dependence on AI inference for basic mapping. In the implemented system, this behavior is supported by seed food records, alias map generation, and parser-to-meal-item transformation logic.

FR-04 Complete Meal Lifecycle Management: The system shall provide full meal lifecycle operations so users can trust and maintain data quality over time. Users shall be able to create meals, update meals, and delete meals, and

each operation shall correctly update day-level aggregates. Edits may include source text changes, metadata updates, or item-level replacement. Deletion shall remove meal-linked items through relational cascade behavior and trigger summary recomputation. This requirement prevents drift between logged entries and dashboard output and ensures the product remains usable beyond initial data capture.

FR-05 History Retrieval with Filtering and Pagination: The system shall allow retrieval of past meal entries by date range and pagination controls to support review and correction workflows. The API shall provide total count with page slices to support client-side navigation. This requirement maps to `GET /api/v1/meals/history` with `start_date`, `end_date`, `limit`, and `offset` parameters. User value comes from auditability: users can revisit previous days, verify estimates, and correct data where needed.

FR-06 Day-Level Dashboard Aggregation: The system shall compute and expose daily nutrition summaries for calories, protein, carbohydrates, fat, and fibre. For each macro, the dashboard response shall include consumed, target, and remaining values. In addition, day-level meal count and streak days shall be included to represent logging consistency. This requirement maps to `GET /api/v1/dashboard/today` and the summary recomputation service. The product intent is immediate feedback after every logging action, reducing user mental math and reinforcing next-step decisions.

FR-07 User Goal Initialization and Persistence: The system shall maintain per-user macro targets used in dashboard evaluation. If explicit goal data is missing for a user, the system shall initialize defaults to keep the workflow unblocked. Goal values shall be persisted and used consistently in remaining-value calculations. This requirement ensures every dashboard response is contextualized and prevents null-target behavior in new-user flows.

FR-08 Context-Aware Nutrition Chat: The system shall provide a chat endpoint that accepts user questions and responds with concise, practical suggestions informed by day-level nutrition context. The chat layer shall access current summary values and detect major intake gaps (for example, protein or fibre deficit, calorie overshoot) before generating response text. The requirement does not demand advanced personalized medical counseling; it requires lightweight actionable assistance that complements the logging/dashboard loop.

FR-09 Chat and Interaction Record Persistence: The system shall store user and assistant chat messages with role and context day for continuity and future analysis. This enables traceability, future conversational UX improvements, and integration with personalization modules in later phases. Storage must remain tied to user identity to preserve isolation across user sessions.

FR-10 Consistent User Context Across Endpoints: All core endpoints shall operate against a resolved user identity so that meals, summaries, goals, and chat records are correctly partitioned per user. In the current implementation this is provided through bearer-token authentication (`Authorization: Bearer <access_token>`) resolved by `get_current_user/get_user_id`, with automatic user and goal bootstrap where missing. This requirement is foundational for data integrity.

FR-11 Input Validation and Error Signaling: The system shall validate request payloads and return clear error responses for invalid input, missing entities, and unsupported operations. Example behaviors include `400` for invalid meal parse/update payloads and `404` for missing meal IDs on update/delete. This requirement ensures predictable client behavior and improves debuggability during integration and future platform hardening.

FR-12 Frontend Workflow Coverage for Backend Capabilities: The user interface shall expose all critical backend operations required in the primary loop: meal logging, same-day dashboard viewing, history-based edit/delete, and chat interactions. This requirement is satisfied by the Streamlit interface with Today, History, Chat, and Settings tabs. The requirement exists because backend specialization still demands demonstrable end-user flow continuity, not isolated API correctness.

B. Non-Functional Requirements

NFR-01 Performance and Responsiveness: The primary user action (meal logging) should return quickly enough to preserve interaction momentum. Target behavior is sub-2-second responses in typical local or low-latency deployments for common inputs. Dashboard retrieval should feel near-instant for single-day queries. Performance optimization should prioritize parser and summary paths because they sit on the highest-frequency journey.

NFR-02 Reliability of Core Flows: The system shall remain operational for create/read/update/delete meal operations and day-level summary generation under normal usage conditions. Failures in one request should not corrupt stored data for previous requests. Transactional boundaries around meal write operations and summary recomputation should preserve consistency. Reliability in this context means users can trust that visible dashboard values match stored meal states after each operation.

NFR-03 Data Consistency and Traceability: The system shall preserve both raw input (`original_text`) and normalized item outputs so generated nutrition values are explainable. Confidence scores and assumption metadata shall be retained for transparency and future parser improvements. Unique constraints and relational integrity must prevent duplicate or orphan summary records for a given user/day combination.

NFR-04 Usability and Low Cognitive Load: The product shall maintain minimal interaction complexity across core workflows. Users should not need to navigate deep menus or perform extensive manual data entry to log a meal and view progress. Interfaces should prioritize clear labels, concise metrics, and straightforward correction controls. Usability is measured here as completion ease of everyday tasks rather than visual sophistication.

NFR-05 Maintainability and Modularity: Code organization shall support extension and testing by separating concerns across API routing, service logic, schema models, and data models. Business logic should remain concentrated in services rather than endpoint controllers. This requirement reduces future refactor cost and supports incremental features such as improved parsing, richer recommendations, and infrastructure migration.

NFR-06 Scalability Readiness: Even though the current implementation targets local development with SQLite, the

design shall remain compatible with migration to production-grade data stores and deployment environments. Data model structure, endpoint contracts, and modular services should permit movement to PostgreSQL, managed caching, and cloud runtime without redesigning the product loop.

NFR-07 Security Baseline: The system shall enforce user-scoped data access at the application level and validate all incoming payloads. Sensitive runtime configuration shall be externalized through environment variables. The current implementation includes token-based authentication with token expiry and revocation support, and remains compatible with further hardening such as stronger secret management and rate limiting.

NFR-08 Cost-Aware Intelligence: AI-assisted behavior should remain lightweight and context-driven, avoiding high-cost model invocation patterns for simple deterministic tasks. Parsing and mapping should prefer deterministic methods first, with AI kept as supplementary support logic where needed. This requirement ensures the architecture remains economically viable as request volume grows.

NFR-09 Observability and Error Visibility: The system should expose enough runtime information to diagnose failures and performance bottlenecks in key flows. At minimum, endpoint-level errors should be surfaced clearly to clients, and server-side logging should support issue tracing for parser behavior, summary recomputation, and chat generation paths.

NFR-10 Testability and Verification: Critical business functions should be testable in isolation, especially parser behavior, macro aggregation, and endpoint contracts. Integration tests should verify that meal CRUD actions correctly propagate to dashboard outputs. Requirement verification should include both software correctness and user-flow correctness to match product goals.

Requirement Assumptions and Constraints: The current requirement set assumes non-clinical usage, where macro values are indicative rather than medically prescriptive. It assumes users can provide simple text meal descriptions and are willing to correct occasional estimation errors. It also assumes backend-first development priority with pragmatic UI support rather than polished production UX in this phase. Infrastructure assumptions include local-first development with future migration pathways.

Acceptance Orientation for This Phase: A requirement in this phase is considered satisfied when the related flow is demonstrable end to end through the implemented frontend and API stack, returns predictable responses, preserves user-scoped data integrity, and updates dashboard context correctly after meal mutations. Accuracy is evaluated as practical usefulness for daily decisions, not laboratory precision.

Overall, the requirement gathering output defines NutriFlow as a focused, behavior-oriented nutrition system. The functional requirements ensure the product can capture, structure, aggregate, and explain daily intake. The non-functional requirements ensure that these capabilities remain fast, reliable, understandable, and extensible. Together they establish a coherent baseline for the next report sections on architecture, database design, and feature implementation detail.

III. System Architecture

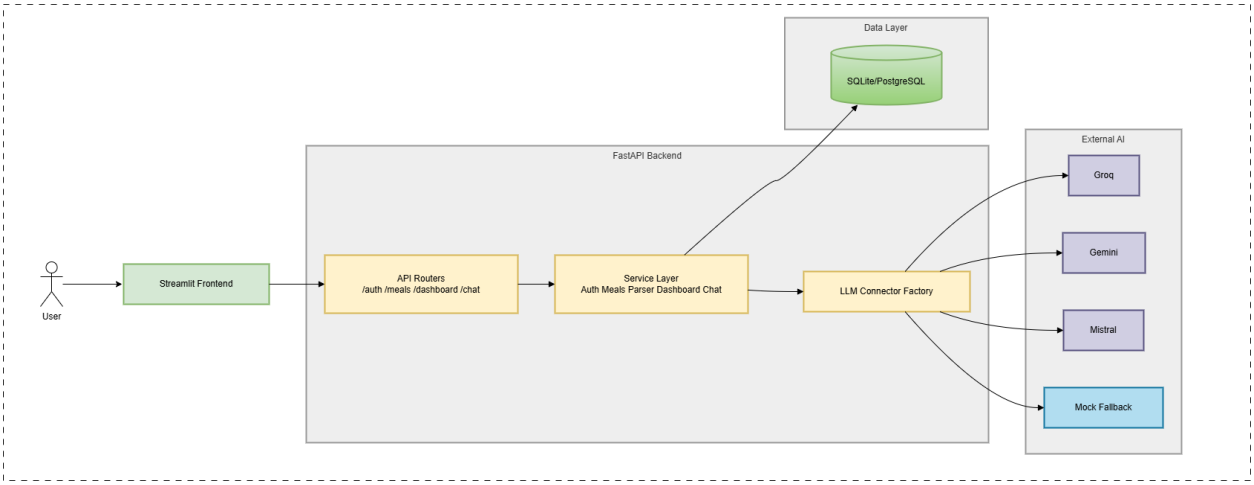

Fig. 3 High-level component architecture showing user interaction through Streamlit frontend, FastAPI backend modules, service layer, data store, and external LLM connectors.

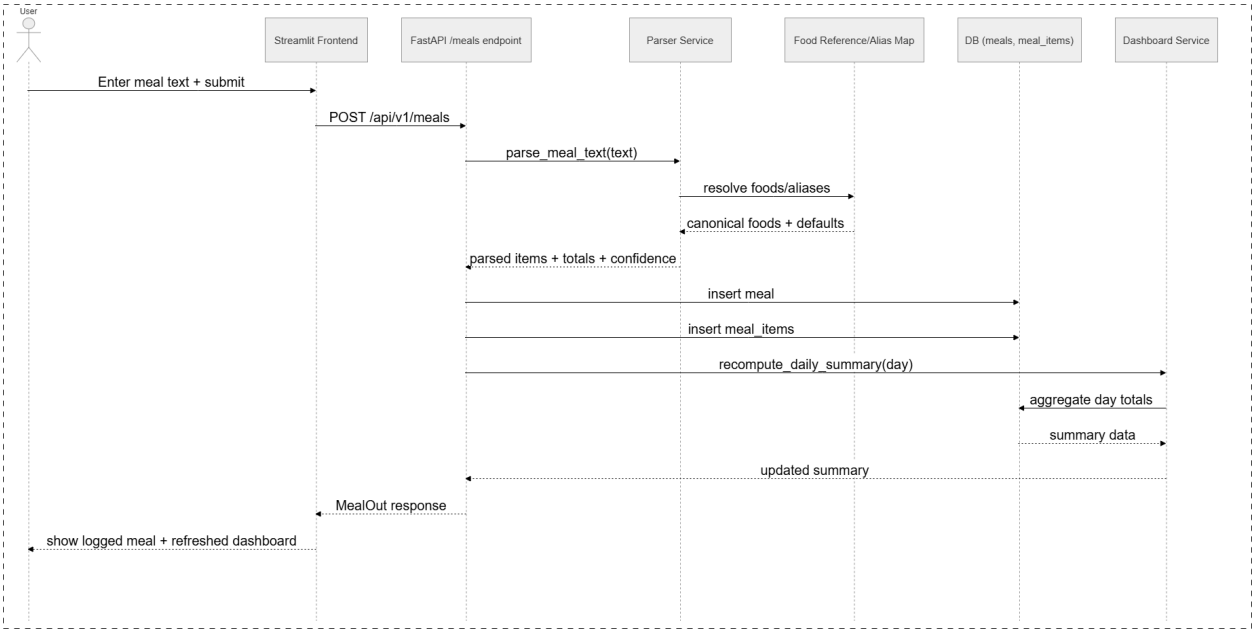

Fig. 4 Meal logging sequence from user input through parsing, food resolution, persistence, summary recomputation, and dashboard refresh response.

NutriFlow is implemented as a backend-centered, modular monolith architecture designed for low-friction feature delivery, clear service boundaries, and straightforward evolution to production infrastructure. The architecture was selected to support the core product loop at high reliability: natural-language meal logging, daily macro aggregation, and contextual nutrition guidance. Instead of prematurely splitting the system into multiple distributed services, the project keeps domain responsibilities separated inside a single FastAPI application and a single relational database. This allows

strong transaction consistency for meal operations while preserving maintainability through explicit module boundaries.

At a high level, the architecture consists of five layers: presentation layer, API orchestration layer, domain service layer, persistence layer, and operational configuration layer. The presentation layer is a Streamlit client that captures user intent and renders dashboards, history, and chat. The API layer exposes versioned HTTP endpoints and handles validation, dependency resolution, and error translation. The domain service layer performs parsing, food mapping, macro computation, summary recomputation, and chat response generation. The persistence layer stores users, goals, foods, meals, meal items, summaries, and chat messages using SQLAlchemy models. The operational layer handles runtime configuration and initialization concerns such as database bootstrap, environment variables, and health checks.

A. Architectural Goals and Constraints

The architecture was shaped by four product goals and three implementation constraints.

Product goals:

1) Minimize logging friction by accepting unstructured meal input.
2) Provide immediate and readable feedback after each logging action.
3) Preserve user trust by allowing correction and history review.
4) Keep assistance contextual, short, and operationally affordable.

Implementation constraints:

1) Backend specialization focus with explicit API and service depth.
2) End-to-end demonstrability with a practical client, not just isolated APIs.
3) Scope discipline to avoid feature sprawl in early versions.

These constraints favor a modular monolith. A microservice architecture would increase operational complexity through service discovery, network-level retries, distributed tracing, and multi-service deployment coordination, without adding meaningful value at current scale. A tightly coupled script-style architecture, on the other hand, would reduce maintainability and make requirement-to-code traceability weak. The selected architecture intentionally sits between these extremes.

B. System Context and External Interfaces

NutriFlow currently operates as a single-process runtime with user scoping enforced at the API dependency layer. The frontend communicates with the backend over HTTP and sends `Authorization: Bearer <access_token>` for protected routes. The backend exposes a stable API prefix (`/api/v1`) and routes requests to feature modules. The database is accessed through SQLAlchemy session dependency injection. In local mode, SQLite is used; the architecture remains portable to PostgreSQL by design since model definitions and query patterns are database-agnostic for primary operations.

External interface types are:

1) Human-to-system input: free-text meals, dashboard day selection, chat prompts, and history filters.
2) Client-to-server API calls: REST requests using JSON payloads.
3) Server-to-database calls: ORM-based select/insert/update/delete and aggregation queries.
4) Operational interface: health endpoint and application startup initialization.

The frontend is deliberately thin in business logic. It delegates all core calculations and parsing decisions to the backend, ensuring single-source truth for behavior and reducing client inconsistency risks.

C. Logical Component Architecture

The backend component structure is organized by responsibility:

1) `app.main`: application composition root. Creates FastAPI app, registers API router, and initializes database metadata at startup.
2) `app.api.v1.router`: endpoint aggregation layer. Mounts `health`, `meals`, `dashboard`, and `chat` routers.
3) Endpoint modules (`endpoints/*.py`): request orchestration layer. Resolve dependencies, invoke services, and map domain exceptions to HTTP responses.
4) Service modules (`services/*.py`): domain logic layer. Contains parser, meal workflows, dashboard aggregation, user bootstrap, food reference mapping, and chat response construction.
5) Schema modules (`schemas/*.py`): contract layer. Pydantic request/response models define payload shape, validation, and serialization behavior.
6) Model modules (`db/models/*.py`): persistence structure. SQLAlchemy entities define tables, constraints, and relationships.
7) Core modules (`core/*.py`): cross-cutting runtime concerns. Configuration and database session setup live here.

This decomposition makes behavior discoverable and testable. Endpoint handlers remain thin and deterministic, while business invariants are centralized in services. This is important for report traceability because requirements can be mapped from user story to endpoint to service to database writes.

D. Request Lifecycle Architecture

A typical request flow follows this sequence:

1) Frontend action triggers `BackendClient` request with JSON body and bearer token header for protected routes.
2) FastAPI router matches route and validates payload against Pydantic schema.
3) Dependency injection resolves database session and user identifier.
4) Endpoint ensures user and goal records exist (`ensure_user_and_goal`).
5) Endpoint delegates to domain service.

6) Service executes business logic and database interactions.

7) Endpoint returns typed response or HTTP error if raised.

8) Frontend updates UI state based on response and optionally re-renders.

This lifecycle is consistent across meals, dashboard, and chat. Uniformity lowers cognitive load for future development and reduces variation in error handling behavior.

E. Meal Logging Pipeline Architecture

Meal logging is the most important architectural path. It has the highest frequency and directly affects user retention.

The flow for `POST /api/v1/meals` is:

1) Validate request (`text` required, optional `meal_type`, optional `eaten_at`).

2) Ensure seed food references exist for first-run database state.

3) Build alias map from canonical foods and aliases.

4) Parse meal text into chunks.

5) Infer quantities using numeric and lexical tokens.

6) Resolve food matches using exact alias map and fallback partial match.

7) Compute item-level macros from food defaults and quantity factors.

8) Apply unknown-item fallback when food is not resolved.

9) Persist meal record and associated item records.

10) Recompute daily summary for the affected day.

11) Return meal with totals and parse confidence.

This pipeline combines deterministic parsing with controlled fallbacks. Unknown foods are not hard failures by default; instead, they are stored with transparent assumptions and lower confidence. This architecture supports the product requirement of preserving flow continuity under imperfect user input. It also enables future enhancements because assumptions are persisted per item and can be used to improve mappings or suggest corrections later.

The parser architecture itself has three stages:

1) Normalization stage: trim, lowercase, collapse whitespace, split on delimiters such as comma and "and".

2) Interpretation stage: extract quantity token and body phrase, normalize quantity from either numeric string or known words.

3) Resolution stage: map phrase to food reference, calculate nutrition values, assign confidence and assumptions.

The parser returns a `ParsedMeal` aggregate with item list and average confidence. This design keeps parsing side-effect free until service orchestration persists results.

F. Meal Mutation and Consistency Model

Update and delete workflows are architected with consistency as a first-class requirement.

For update (`PATCH /api/v1/meals/{meal_id}`):

1) Resolve meal by `meal_id` and `user_id`.
2) Track old day for summary recomputation.
3) Apply metadata updates (`meal_type`, `eaten_at`) if present.
4) If `text` is provided, re-run parse pipeline and replace all items.
5) If manual items are provided, validate canonical names, map to foods, and replace all items.
6) Commit transaction.
7) Recompute summaries for old day and new day (if date changed).
8) Return updated meal aggregate.

For delete (`DELETE /api/v1/meals/{meal_id}`):

1) Resolve meal by `meal_id` and `user_id`.
2) Capture affected day.
3) Delete meal (with cascade delete for child items).
4) Commit transaction.
5) Recompute summary for affected day.

This model prevents stale dashboard values after mutation. Recomputing summaries post-commit ensures the analytical view remains aligned with transactional writes. The architecture trades small extra query cost for stronger correctness, which is acceptable at current scale and aligns with user trust priorities.

G. Dashboard Aggregation Architecture

The dashboard subsystem provides the reinforcement loop for user behavior. Its architecture is intentionally deterministic and query-efficient.

Core functions:

1) `recompute_daily_summary`: aggregate meals and items for user/day into materialized `daily_summaries` row.
2) `get_streak_days`: scan backward from day until no-meal summary found.
3) `get_today_dashboard`: combine consumed totals with user goals into macro pairs.

Aggregation query design:

1) Join `meals` with `meal_items`.
2) Filter by user and day boundaries (`time.min` to `time.max`).
3) Use SQL aggregates for meal count and macro sums.

4) Upsert-like behavior on daily summary entity.

Response model architecture exposes a normalized contract (`MacroPair`) for each macro with `consumed`, `target`, and `remaining`. This keeps frontend rendering simple and consistent, and it decouples display logic from storage schema.

Materializing daily summaries is an architectural decision that improves read performance and simplifies streak computation. Instead of recomputing historical days every time, the system updates one day per mutation, then reads directly. This is a suitable middle ground between pure on-the-fly aggregation and complex event-driven projection infrastructure.

H. Chat Guidance Architecture

Chat is built as a context-enriched response service rather than a full conversational AI platform. The architecture focuses on utility, cost control, and predictability.

Flow for `POST /api/v1/chat`:

1) Validate prompt and optional context day.

2) Resolve user context and ensure user/goal records.

3) Fetch dashboard summary for context day.

4) Build response text using macro gap heuristics and prompt intent signals.

5) Persist user and assistant messages.

6) Return response and context day.

The response engine uses deterministic rules:

1) Detect nutrient deficits or calorie overshoot.

2) Format a short status line.

3) Add suggestion template based on prompt keywords.

4) Append behavior cue encouraging continued logging.

This architecture deliberately avoids heavy model orchestration for everyday replies. It is extensible: a future personalization or model-inference layer can replace `_build_response` without changing API contracts, database schema, or frontend interaction structure.

I. Data Layer and Entity Relationship Architecture

Persistence uses SQLAlchemy declarative models with explicit relations:

1) `users` as root identity entity.

2) `user_goals` one-to-one target profile.

3) `foods` canonical nutrition reference.

4) meals parent record for each log entry.

5) meal_items child rows for normalized meal components.

6) daily_summaries one row per user/day aggregate.

7) chat_messages chronological chat history.

Key relationship design:

1) users to meals is one-to-many with cascade delete.

2) meals to meal_items is one-to-many with delete-orphan semantics.

3) foods to meal_items is optional many-to-one (SET NULL on food delete).

4) users to daily_summaries is one-to-many with unique constraint on (user_id, day).

5) users to chat_messages is one-to-many.

This model supports both transactional and analytical operations without duplication of source-of-truth entities. Raw text is stored in meals for auditability and parser reprocessing; itemized data enables macro rollups and future feature extensions.

J. API Contract Architecture

API contract design follows versioned REST patterns with explicit schemas:

1) POST /api/v1/meals

2) PATCH /api/v1/meals/{meal_id}

3) DELETE /api/v1/meals/{meal_id}

4) GET /api/v1/meals/history

5) GET /api/v1/dashboard/today

6) POST /api/v1/chat

7) GET /api/v1/health

Schema architecture principles:

1) Input constraints prevent malformed payloads (length, optionality, numeric ranges).

2) Output models are strongly typed and domain-aligned.

3) Validation errors are surfaced early at boundary.

4) Endpoint responses are predictable for UI consumption.

Error architecture:

1) Domain ValueError is mapped to 400.

2) Missing meal references map to 404.

3) Network or transport errors are handled client-side in ApiError.

This contract architecture provides clean separation between transport concerns and domain concerns. It also

stabilizes integration as frontend and backend evolve independently.

K. Frontend Integration Architecture

The Streamlit client is organized around view modules and a small API client wrapper.

Frontend components:

1) `app.py` as shell and tab composition.
2) `client.py` as request abstraction with status validation.
3) `state.py` for session defaults and error state.
4) `ui_today.py` for logging and dashboard.
5) `ui_history.py` for filtered history and CRUD actions.
6) `ui_chat.py` for conversational interaction.
7) `ui_settings.py` for backend URL and user ID control.

Architectural rationale:

1) Keep business logic server-side.
2) Keep UI state explicit and session-scoped.
3) Provide consistent error feedback patterns.
4) Avoid duplicate macro calculations in frontend.

This architecture makes frontend a thin orchestration and visualization layer. It allows quick iteration and keeps behavioral correctness concentrated in backend services where requirements are enforced.

L. Cross-Cutting Concerns

Configuration architecture:

1) `Settings` dataclass reads environment variables for app name, API prefix, and database URL.
2) Database engine is initialized once with SQLite-specific connection args when needed.
3) Session lifecycle is handled per request via dependency generator.

Initialization architecture:

1) Application startup triggers `init_db`.
2) All mapped tables are created from metadata.
3) Seed food data is inserted lazily on first meal path.

User context architecture:

1) `get_token` extracts bearer credentials and `get_current_user` validates token state.
2) `get_user_id` derives identity from the authenticated user object.
3) `ensure_user_and_goal` guarantees user graph exists before business action.

Health and operations:

1) Root endpoint confirms service availability.

2) Health endpoint returns simple status for orchestration and monitoring.

These cross-cutting choices keep local developer setup minimal and align with the project goal of demonstrating complete backend functionality without heavy infrastructure prerequisites.

M. Quality Attributes and Trade-Off Analysis

Performance:

1) Deterministic parser path avoids expensive inference overhead for common logs.

2) Daily summary materialization avoids repeated deep aggregations for dashboard views.

3) Remaining bottlenecks are bounded to meal write and summary recompute path.

Reliability:

1) Transactional commits ensure writes are atomic per operation.

2) Summary recompute on every mutation favors correctness over minimal compute.

3) Clear error mappings make failure modes predictable.

Scalability:

1) Current monolith supports moderate scale with vertical resources.

2) Service boundaries already prepared for future extraction if needed.

3) Database portability enables migration from SQLite to PostgreSQL.

Maintainability:

1) Layered separation improves readability and unit-test potential.

2) Typed schemas reduce integration ambiguity.

3) Domain logic centralization prevents endpoint-level duplication.

Security posture:

1) Protected routes enforce bearer-token auth with expiry and revocation checks.

2) Configuration externalization supports secure secret handling in production.

3) Validation and bounded payload lengths reduce abuse vectors.

The architecture consciously optimizes for product momentum and behavior-loop integrity rather than distributed-system sophistication. This is appropriate for the current phase and aligns with requirement priorities.

N. Extensibility Architecture

The current design keeps extension points explicit:

1) Parser extension: plug improved alias resolution, fuzzy matching, or model-assisted parsing.

2) Recommendation extension: replace deterministic chat builder with policy/model layers.

3) Data extension: add weekly summaries or trend tables without changing core meal schema.

4) API extension: add goal management endpoints while preserving existing contracts.

5) Infra extension: introduce Redis caching for alias maps and hot dashboard reads.

Future intelligent modules can be introduced behind service interfaces first, and only split into separate deployable services when scale justifies network and orchestration complexity.

O. End-to-End Architectural Narrative

A representative user journey demonstrates architectural cohesion:

1) User logs "2 roti, dal, curd" in Today tab.

2) Frontend posts payload to meals endpoint.

3) Backend validates, resolves user, parses items, computes macros, stores records.

4) Backend recomputes daily summary and returns meal output.

5) Frontend refreshes dashboard and displays updated remaining macros.

6) User edits a meal in History tab; backend replaces items and recomputes both affected day summaries.

7) User asks chat for dinner suggestion; backend reads day context and returns concise guidance.

Every step in this journey is serviced by a dedicated module with explicit contracts. There are no hidden side channels or duplicated calculations across layers. That architectural consistency is the key strength of the current system.

P. Architectural Conclusion

NutriFlow's system architecture is intentionally focused, modular, and requirement-driven. The modular monolith approach delivers a strong balance of speed, correctness, and clarity for a backend specialization project. By keeping parsing, aggregation, and guidance logic in well-scoped services, and by enforcing typed API contracts over a stable data model, the system achieves end-to-end coherence for the product's central behavior loop. The architecture is production-aware without being over-engineered, and it is extensible enough to support future enhancements such as richer food intelligence, personalized recommendations, stronger authentication, and cloud-scale deployment patterns.

IV. Database Design

The NutriFlow database is designed to support a behavior-first nutrition tracking workflow where fast meal capture, transparent estimation, daily aggregation, and correction-friendly history are all first-class requirements. The schema intentionally balances transactional integrity with analytical readability. Instead of building separate operational and warehouse databases at this stage, the design uses a single relational schema that can serve both CRUD-heavy logging operations and day-level dashboard summarization. This is appropriate for the project scope because it keeps consistency

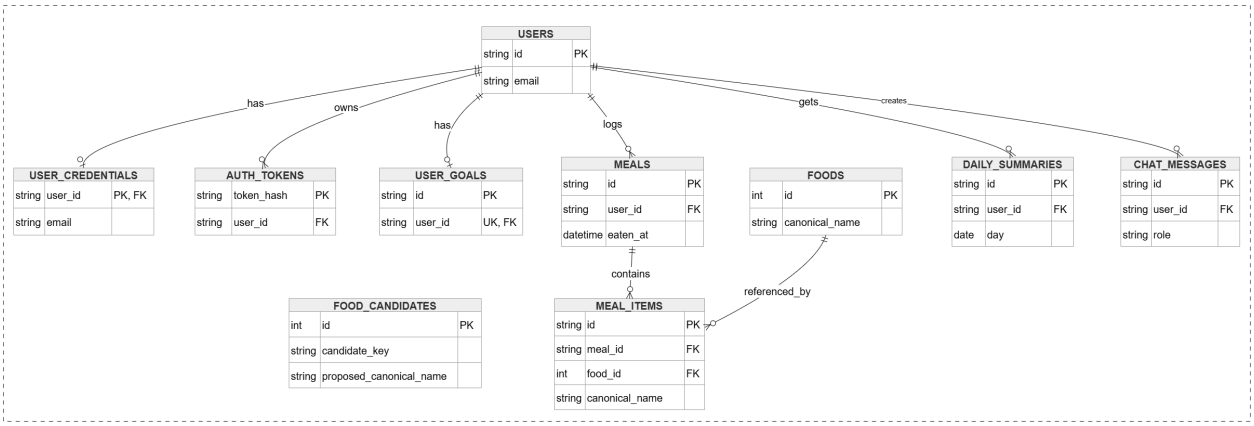

Fig. 5 Brief ER diagram showing core entities and key relationships used for authentication, meal logging, aggregation, and chat context.

high, reduces orchestration complexity, and preserves end-to-end traceability between user input and computed nutrition outputs.

From an implementation standpoint, the current system uses SQLAlchemy declarative models and defaults to SQLite for local execution. However, entity structure, constraints, and query patterns are intentionally portable to PostgreSQL for production deployment. The schema is normalized around a clear domain model: users own goals, meals, summaries, and chat messages; meals contain itemized food entries; food reference data provides canonical nutrient values and alias mapping. This model directly reflects product requirements: log quickly, compute macros consistently, view daily progress, and correct entries when needed.

A. Data Design Principles

The database architecture is guided by the following principles:

1) Preserve source and interpretation together: raw user meal text must be stored alongside parsed item rows so estimates are explainable and editable.
2) Separate reference and event data: canonical foods are maintained independently from user meal events to avoid duplication and simplify mapping updates.
3) Keep user context explicit: every mutable user-generated record is keyed by `user_id` to enforce logical tenancy.
4) Materialize frequent aggregates: day-level summaries are stored to speed dashboard reads and simplify streak computation.
5) Support correction workflows: schema and relationships must allow update/delete operations to safely propagate to aggregates.
6) Prioritize deterministic integrity: foreign keys, uniqueness constraints, and cascade policies enforce consistency without requiring complex application-side reconciliation.

B. Logical Data Model Overview

The schema consists of seven core tables:

1) `users`
2) `user_goals`
3) `foods`
4) `meals`
5) `meal_items`
6) `daily_summaries`
7) `chat_messages`

The design pattern is a mixed master-reference and event model:

1) Master entities: `users`, `user_goals`, `foods`
2) Event entities: `meals`, `meal_items`, `chat_messages`
3) Derived/materialized entity: `daily_summaries`

This split is deliberate. Master entities change slowly and define context. Event entities capture user actions over time. Derived entity accelerates high-frequency read paths (dashboard and streak).

C. Table Specification and Rationale

1. `users`

Purpose: root identity anchor for all user-scoped data.

Key fields:

1) `id` (String(36), primary key): UUID-like identifier used as cross-table reference.
2) `email` (String(255), unique, nullable): optional contact field for future identity expansion.
3) `created_at` (DateTime): audit timestamp.

Rationale:

1) Even in local mode using token-authenticated user resolution, storing a concrete user entity preserves relational integrity.
2) Keeping `email` nullable allows frictionless local testing and phased onboarding implementation.
3) This table acts as the parent for goals, meals, summaries, and chat messages.

Relationship role:

1) One-to-many with `meals`.
2) One-to-many with `daily_summaries`.
3) One-to-many with `chat_messages`.
4) One-to-one logical with `user_goals` (enforced by unique key on `user_goals.user_id`).

2. user_goals

Purpose: store per-user nutrition targets used for dashboard comparisons.

Key fields:

1) `id` (String(36), primary key)
2) `user_id` (String(36), foreign key to `users.id`, unique, non-null)
3) `calories_target` (Float, default 2000)
4) `protein_target` (Float, default 120)
5) `carbs_target` (Float, default 250)
6) `fat_target` (Float, default 60)
7) `fibre_target` (Float, default 30)
8) `updated_at` (DateTime)

Rationale:

1) Goals are decoupled from user profile to support future goal history/versioning if needed.
2) Unique `user_id` enforces one active goal profile per user in current scope.
3) Defaults allow immediate usability for first-time users and prevent null-target dashboards.

Design note:

1) The service layer bootstraps goals via `ensure_user_and_goal`, ensuring summary generation is never blocked by missing targets.

3. foods

Purpose: canonical nutrition reference and alias resolution base.

Key fields:

1) `id` (int, primary key, autoincrement)
2) `canonical_name` (String(120), unique, non-null)
3) `aliases_csv` (String(500), non-null, default empty)
4) `default_quantity` (Float, non-null)
5) `default_unit` (String(40), non-null)
6) Per-serving nutrition fields: `calories_per_serving`, `protein_per_serving`, `carbs_per_serving`, `fat_per_serving`, `fibre_per_serving` (Float)

Rationale:

1) Keeps nutritional reference independent from user events for reuse and consistency.
2) `canonical_name` uniqueness prevents duplicate base foods.
3) Alias storage supports colloquial/variant food terms during parsing.

4) Default quantity/unit support graceful handling of missing user quantities.

Trade-off:

1) Aliases are currently stored as CSV for implementation simplicity.
2) For larger scale, alias normalization into separate `food_aliases` table could improve query flexibility and data governance.

4. meals

Purpose: represent a single meal logging event by a user.

Key fields:

1) `id` (String(36), primary key)
2) `user_id` (String(36), foreign key to `users.id`, indexed)
3) `original_text` (Text, non-null)
4) `meal_type` (String(40), nullable)
5) `eaten_at` (DateTime, non-null)
6) `parse_confidence` (Float, non-null)
7) `created_at` and `updated_at` (DateTime)

Rationale:

1) `original_text` preserves user-provided source for auditability and reprocessing.
2) `parse_confidence` stores aggregate quality signal from parser output.
3) `eaten_at` decouples meal time from creation time, enabling historical entry and correct day-level aggregation.
4) `meal_type` is nullable to avoid forcing user classification in low-friction flows.

Relationship behavior:

1) Parent of `meal_items` with cascade delete-orphan.
2) Belongs to `users` with cascade on user delete.

5. meal_items

Purpose: normalized food-item components generated from meal parsing or manual item updates.

Key fields:

1) `id` (String(36), primary key)
2) `meal_id` (String(36), foreign key to `meals.id`, indexed)
3) `food_id` (nullable foreign key to `foods.id`, `SET NULL` on delete)
4) `raw_label` (String(160), non-null)
5) `canonical_name` (String(120), non-null)

6) `quantity` (Float, non-null)

7) `unit` (String(40), non-null)

8) Computed nutrition fields: `calories`, `protein`, `carbs`, `fat`, `fibre` (Float)

9) `confidence` (Float, non-null)

10) `assumptions` (JSON, non-null, default dict)

11) `created_at` (DateTime)

Rationale:

1) Item-level granularity enables detailed totals and correction-friendly workflows.

2) `food_id` nullable design allows unknown-food fallbacks while preserving a valid item row.

3) `assumptions` JSON captures parser decisions (for example defaulted quantity or unknown mapping), supporting transparency and future model improvement.

4) Nutrition values are denormalized into row fields for fast aggregation and historical consistency, even if reference food values change later.

Important design choice:

1) `canonical_name` is stored even when `food_id` exists, creating a stable display field and reducing join dependency in read-heavy paths.

6. daily_summaries

Purpose: materialized per-user, per-day macro totals and meal count.

Key fields:

1) `id` (String(36), primary key)

2) `user_id` (String(36), foreign key to `users.id`, indexed)

3) `day` (Date, non-null)

4) Consumed totals: `calories_consumed`, `protein_consumed`, `carbs_consumed`, `fat_consumed`, `fibre_consumed` (Float)

5) `meal_count` (Integer, non-null)

6) `updated_at` (DateTime)

Constraint:

1) UniqueConstraint on (`user_id`, `day`) ensures exactly one summary row per user/day.

Rationale:

1) Dashboard is a frequent read; pre-aggregated daily data prevents repeated heavy joins.

2) Streak computation becomes straightforward by checking consecutive summary rows with meal_count > 0.

3) Summary rows are recomputed after meal create/update/delete, ensuring deterministic consistency.

7. chat_messages

Purpose: persist conversational context and assistant outputs for each user.

Key fields:

1) `id` (String(36), primary key)
2) `user_id` (String(36), foreign key to `users.id`, indexed)
3) `role` (String(20), non-null)
4) `content` (Text, non-null)
5) `context_day` (Date, nullable)
6) `created_at` (DateTime)

Rationale:

1) Maintains interaction history and supports future personalization or analytics.
2) `context_day` links responses to nutritional context used by the assistant.
3) Separate storage for chat avoids polluting meal tables with conversational metadata.

D. Relationship and Cardinality Design

Primary cardinalities:

1) `users` 1:N `meals`
2) `users` 1:N `daily_summaries`
3) `users` 1:N `chat_messages`
4) `users` 1:1 `user_goals` (logical one-to-one)
5) `meals` 1:N `meal_items`
6) `foods` 1:N `meal_items` (optional link)

Why this structure works:

1) User ownership remains explicit at top-level event entities.
2) Meal decomposition supports both raw and normalized views.
3) Optional food reference link allows resilient parsing under uncertainty.
4) Daily summary table decouples aggregate reads from event volume.

Cascade policy decisions:

1) Deleting user cascades to meals, summaries, goals, and chat for clean tenant removal.
2) Deleting meal cascades to meal_items (no orphaned items).
3) Deleting food sets `meal_items.food_id` to null, preserving historical nutrition records.

This cascade strategy protects history fidelity while maintaining referential integrity.

E. Integrity Constraints and Validation Alignment

Database-level controls:

1) Primary keys on all tables.
2) Foreign keys for relationship integrity.
3) Unique constraints on `foods.canonical_name`, `users.email`, `user_goals.user_id`, and (`daily_summaries.user_id`, `day`).
4) Non-null constraints on essential operational fields.

Application-level validation complements database constraints:

1) Pydantic enforces meal text length and manual item quantity > 0.
2) Empty manual item list is rejected at schema layer.
3) Endpoint/service logic maps invalid operations to clear HTTP errors.

The combined approach avoids overloading either layer. Database ensures structural truth; application ensures domain-meaningful inputs.

F. Indexing and Query Path Considerations

Current explicit indexes exist on high-frequency foreign key fields:

1) `meals.user_id`
2) `meal_items.meal_id`
3) `daily_summaries.user_id`
4) `chat_messages.user_id`

These align with dominant query shapes:

1) Fetch user meals in date ranges for history.
2) Join meal items by meal for totals and details.
3) Fetch/update summary by user and day.
4) Load user chat history in future expansions.

Additional optimization candidates for production:

1) Composite index on `meals(user_id, eaten_at)` for history and day filters.
2) Composite index on `daily_summaries(user_id, day)` to complement uniqueness constraint access path.
3) Optional index on `meal_items(food_id)` if analytical reporting by food is added.

At current scale, the existing schema is adequate; future indexing should be driven by measured query plans and latency metrics.

G. Aggregation Strategy and Consistency Guarantees

A central design choice is materializing daily summaries instead of computing dashboard metrics directly from raw events on every read. The recomputation function executes after meal mutations and writes authoritative totals to `daily_summaries`.

Benefits:

1) Predictable dashboard performance.
2) Simplified streak logic.
3) Lower read-path complexity in API layer.

Consistency model:

1) Meal mutation commits first.
2) Summary recomputation runs for affected day(s).
3) Dashboard reads from recomputed summary and goals.

For date changes during updates, both old and new day summaries are recomputed, preventing ghost counts and stale totals. This explicit two-day reconciliation is critical to correctness.

H. Data Lifecycle and State Transitions

1. Meal creation lifecycle

1) User submits text.
2) Parser returns itemized representation.
3) Meal row inserted.
4) Item rows inserted.
5) Summary recomputed.
6) Response returns meal + totals.

2. Meal update lifecycle

1) Meal located by `meal_id + user_id`.
2) Existing items removed.
3) New parsed/manual items inserted.
4) If `eaten_at` changes date, old and new summaries recomputed.
5) Updated meal returned.

3. Meal deletion lifecycle

1) Meal located by `meal_id + user_id`.

2) Meal deleted with cascading items.

3) Affected day summary recomputed.

This lifecycle-driven schema behavior ensures derived data never drifts far from event source data.

I. Normalization and Controlled Denormalization

Normalized aspects:

1) User identity and goals separated.

2) Food reference separated from event data.

3) Meal and meal_items split to represent 1:N composition.

Denormalized aspects:

1) Nutrition values stored directly in `meal_items`.

2) Daily totals stored in `daily_summaries`.

3) Canonical name copied into meal_items.

Why denormalization is acceptable:

1) Read patterns are aggregate-heavy.

2) Historical accuracy should remain tied to time-of-entry values.

3) Product needs fast feedback over strict storage minimization.

The design is a practical OLTP-plus-light-analytics hybrid appropriate for the application stage.

J. Handling Uncertainty and Approximation in Schema

Nutrition estimation is inherently approximate, especially for home-cooked and mixed meals. The schema explicitly models uncertainty rather than hiding it.

Mechanisms:

1) `parse_confidence` at meal level.

2) `confidence` at meal-item level.

3) `assumptions` JSON to record defaulting/fallback decisions.

4) Optional `food_id` for unknown mappings.

This is an important design strength. It preserves explainability and supports user trust because future UI or reports can communicate estimate confidence and parsing assumptions.

K. Multi-User Isolation and Access Patterns

The schema is single-database, multi-user by key partitioning. Every user-generated event row carries `user_id`, and service queries scope by that key. This enforces logical isolation at query level and aligns with bearer-token

authenticated access.

Practical implications:

1) One user's meal IDs are not visible or mutable by another user when proper scoping is applied.

2) Summaries are computed per user-day boundary.

3) Chat context remains user-specific.

For production hardening, row-level security or strict auth token claims can reinforce this application-level partitioning.

L. Auditability and Traceability

Database design supports auditability through:

1) Timestamps (`created_at`, `updated_at`) on key entities.

2) Raw input retention (`meals.original_text`).

3) Parsed output retention (`meal_items.*`).

4) Assumption metadata storage (`meal_items.assumptions`).

5) Chat record persistence (`chat_messages`).

This traceability is useful for debugging parser behavior, explaining dashboard totals, and demonstrating requirement-to-data linkage in project evaluation.

M. Portability and Migration Readiness

Although local runtime uses SQLite, design choices anticipate migration:

1) UUID-like string keys already portable.

2) SQLAlchemy abstractions avoid engine-specific SQL in core paths.

3) Table relationships and constraints map cleanly to PostgreSQL.

4) JSON column usage in meal assumptions is compatible with richer JSON querying in PostgreSQL.

Migration strategy recommendation:

1) Introduce Alembic for controlled schema versioning.

2) Add production indexes after observing actual load.

3) Promote environment-based DB URL and connection pooling.

4) Run data integrity checks during cutover.

The current schema is stable enough to serve as baseline for this transition.

N. Risks, Limitations, and Improvement Opportunities

Known limitations:

1) `aliases_csv` in a single column can become difficult to manage at scale.
2) Float nutrition values can introduce minor rounding artifacts; decimal types may be considered for stricter accounting.
3) No soft-delete strategy is currently used; hard deletes simplify logic but reduce recovery options.
4) Goal history is not versioned; only current targets are retained.

Improvement opportunities:

1) Add `food_aliases` table for normalized alias management.
2) Add weekly/monthly summary tables for longer-period analytics.
3) Add goal history snapshots to improve retrospective reporting.
4) Introduce background recomputation queue if mutation volume increases substantially.

These improvements can be introduced incrementally without redesigning the core relational model.

O. Database Design Conclusion

The NutriFlow database design is purpose-built for a low-friction nutrition tracking product where transactional correctness, fast daily feedback, and transparent approximation are all required simultaneously. The schema organizes domain data into clear entities with explicit relationships, enforces integrity through keys and constraints, and accelerates common read paths through materialized day summaries. It supports full meal lifecycle operations, preserves auditability from raw input to parsed output, and remains extensible for future personalization and scale. Most importantly, the design reflects the project's core philosophy: prioritize consistent daily use through practical, explainable, and maintainable data architecture.

V. Feature Development

Feature development in NutriFlow was carried out as an iterative backend-first process where each user-facing capability was built as a complete vertical slice: API contract, domain service logic, persistence updates, and frontend integration. This approach ensured that every feature could be demonstrated end to end and validated against real user interaction patterns rather than only unit-level behavior. Because the project objective is low-friction daily nutrition tracking, feature prioritization focused on the shortest path from user intent to meaningful feedback. As a result, development concentrated on six high-impact feature groups: meal logging, parsing and normalization, meal history management, dashboard aggregation, contextual chat guidance, and integrated frontend flows.

The implementation follows a modular monolith style in FastAPI with explicit service modules. This structure allowed parallel progress on feature domains without fragmenting runtime operations. Each feature implementation was designed around three constraints: low interaction cost for the user, deterministic behavior for predictable outputs, and correction paths for approximate nutrition estimates. These constraints shaped both core logic and UX behavior

throughout development.

A. Development Strategy and Vertical Slice Execution

The project was not built as a sequence of isolated backend utilities. Instead, feature development followed vertical slices:

1) Define API request and response schema.
2) Implement core business logic in a service module.
3) Persist domain state changes in SQLAlchemy models.
4) Expose endpoint with dependency wiring and error mapping.
5) Integrate client call in Streamlit frontend.
6) Validate full flow through manual interaction and response checks.

This strategy was selected because product value emerges from full loops, not isolated methods. For example, meal logging is only complete when user text becomes parsed items, macro totals are persisted, summary values are updated, and the frontend reflects the change immediately. Implementing features in complete slices reduced integration surprises and ensured traceability from requirement to behavior.

B. Foundation Features: App Composition, Routing, and Configuration

Before domain features, foundational work established a stable execution surface:

1) FastAPI app creation in `app.main`.
2) API version prefix and router composition in `api/v1/router.py`.
3) Database engine/session setup in `core/database.py`.
4) Runtime settings via environment-backed dataclass in `core/config.py`.
5) Health and root endpoints for service checks.

Although foundational, this layer directly supports feature reliability. Versioned routing ensures new endpoints can be added without contract instability. Centralized database session dependency ensures consistent transaction management across all features. Startup database initialization (`init_db`) removes manual setup friction in local environments, which is especially useful when quickly iterating on feature logic.

C. User Context Bootstrap Feature

NutriFlow currently uses a pragmatic user-context strategy built for fast development and repeatable demos:

1) User identity is established by login/register and bearer token issuance.
2) Protected routes use `get_token` + `get_current_user`; `get_user_id` is derived from authenticated context.
3) `ensure_user_and_goal` ensures both `User` and `UserGoal` records exist before feature execution.

This pattern is a feature enabler, not only a technical shortcut. It guarantees that first-time interactions never fail due to missing profile setup. It also keeps all downstream features (meal logging, dashboard, chat) consistently user-scoped. The bootstrap behavior reduces edge-case branching inside domain services and allows feature logic to assume a valid user-goal graph.

D. Core Feature 1: Natural-Language Meal Logging

Meal logging is the primary product feature and receives the deepest implementation investment. The user submits free-text meal content, optionally with meal type and eaten timestamp. The backend transforms this into normalized item records and nutrition totals.

1. API and Schema Design

Endpoint: `POST /api/v1/meals`

Input model (`MealCreateRequest`):

1) `text` required, length-limited.
2) `meal_type` optional.
3) `eaten_at` optional.

Output model (`MealOut`):

1) Meal identity and metadata.
2) Parse confidence.
3) Itemized rows.
4) Totals across calories/protein/carbs/fat/fibre.

This schema design intentionally returns both summary and detail, enabling the frontend to render immediate high-level feedback while still exposing item-level interpretation for user trust.

2. Service Flow

`create_meal` executes these steps:

1) Ensure food reference seeds exist.
2) Build alias map from `foods`.
3) Parse text with `parse_meal_text`.
4) Reject if no items could be produced.
5) Create meal row with raw text and parse confidence.
6) Insert child meal items derived from parser output.
7) Commit transaction.

8) Reload meal and recompute daily summary for affected day.

9) Return meal with computed totals.

Key development decision: parsing is performed before persistence, and summary recomputation is triggered after commit. This ordering preserves consistent state and avoids stale aggregate reads.

3. UX Integration

In `ui_today.py`, meal logging is implemented via a Streamlit form:

1) Text area for natural-language input.

2) Optional meal type.

3) Optional date/time controls.

4) Submit action with error display.

On success, the UI reruns to fetch updated dashboard and day meals. This gives users near-immediate feedback, which is critical to reinforcing logging behavior.

E. Core Feature 2: Parsing and Normalization Engine

The parser is the engine that makes free-text logging possible. It is deterministic, explainable, and resilient against imperfect input.

1. Parsing Pipeline

Implemented in `services/parser.py`:

1) Normalize and split input into chunks (`_split_chunks`).

2) Extract quantity token and body phrase via regex.

3) Resolve quantity using numeric parse and word map (`one`, `two`, etc.).

4) Attempt exact alias-map match.

5) Attempt simple containment fallback match.

6) If unresolved, create unknown fallback item with lower confidence and assumptions.

7) If resolved, compute macro values by scaling food per-serving values with quantity.

8) Default quantity when missing; record assumption.

9) Return list of `ParsedItem` and average confidence (`ParsedMeal`).

2. Confidence and Assumption Modeling

Two details significantly improve feature quality:

1) Confidence scoring: a. High confidence for direct matches with explicit quantity. b. Lower confidence for defaulted quantity or unknown fallback.

2) Assumption recording: a. Captures defaulted quantity. b. Captures unknown food fallback reason.

This data is persisted in `meal_items.assumptions` and `confidence` fields. It supports future explainability and potential UI surfacing of uncertain interpretations.

3. Why Deterministic Parsing First

The project deliberately avoids immediate heavy model-based parsing in base flow. Deterministic parsing gives:

1) Fast response time.

2) Predictable output behavior.

3) Easier debugging.

4) Lower operational cost.

For current scope, this trade-off is correct because the product goal is habit consistency, not maximal linguistic coverage.

F. Core Feature 3: Food Reference Seeding and Alias Mapping

NutriFlow includes a built-in food reference module (`food_reference.py`) that seeds essential foods and provides alias lookup maps.

1. Seed Strategy

`ensure_seed_foods` inserts default foods only when the table is empty. This allows:

1) Zero manual setup for first run.

2) Stable baseline parsing behavior.

3) Easy extension with additional foods in later phases.

2. Alias Map Construction

`get_food_alias_map` builds a dictionary that maps both canonical names and aliases to `Food` objects.

Benefits:

1) O(1)-style lookup behavior for common entries.

2) Simple support for regional or colloquial naming.

3) Decoupling between user wording and canonical nutrition values.

This feature is small in code size but foundational in user experience, because alias matching directly reduces input rejection and manual correction burden.

G. Core Feature 4: Meal Update Workflow

Approximate systems need correction paths. Meal update was therefore treated as a first-class feature rather than an afterthought.

Endpoint: `PATCH /api/v1/meals/{meal_id}`

Supported update modes:

1) Metadata update (`meal_type`, `eaten_at`).
2) Full text reparse (`text` provided).
3) Manual item replacement (`items` provided).

1. Implementation Details

In `update_meal`:

1) Meal is resolved by `meal_id + user_id`; missing meal returns `None`.
2) Old day is captured to handle date-shift recomputation.
3) Text updates trigger parser rerun and complete replacement of existing items.
4) Manual item updates map canonical names to foods and generate parsed-equivalent item rows.
5) Existing items are removed before replacement to prevent stale totals.
6) Transaction commits.
7) Summaries recomputed for both old and new day.

2. Design Trade-Off

Current update design replaces all meal items rather than performing partial item diffs. This keeps logic simple and deterministic, reducing mutation edge cases in early phase. As scale grows, partial-diff updates can be introduced if performance profiling demands it.

3. Frontend Integration

Both Today and History tabs offer inline edit forms. This duplicated placement is intentional:

1) Users can quickly correct recent logs in same context.
2) Users can correct historical entries via date-filtered history.

The form posts changes to backend and reruns view on success, preserving a simple mental model.

H. Core Feature 5: Meal Deletion Workflow

Deletion is implemented through `DELETE /api/v1/meals/{meal_id}` and returns `204` on success.

Implementation flow:

1) Resolve meal by user scope.

2) Capture affected day.

3) Delete meal row; item rows cascade-delete.

4) Commit and recompute affected summary.

5) Return no content.

Key reliability behavior:

1) Missing meal returns `404`.

2) Frontend handles `404` as already removed and refreshes view.

This prevents confusing UX states when concurrent operations or stale UI references occur.

I. Core Feature 6: Meal History Retrieval with Filters and Pagination

History is a key feature for trust, auditability, and corrections.

Endpoint: `GET /api/v1/meals/history`

Parameters:

1) `start_date` optional.

2) `end_date` optional.

3) `limit` with range guard.

4) `offset` with non-negative guard.

Response:

1) `items` list of simplified meal entries.

2) `total` total count for navigation.

1. Query Design

`get_meal_history` composes a SQLAlchemy query scoped by user and optional date boundaries. It calculates total count via subquery and retrieves ordered page slices by `eaten_at desc`.

Why this design matters:

1) Supports responsive browsing in growing data volume.

2) Prevents loading all records into memory.

3) Enables deterministic pagination behavior in client.

2. UI Behavior

History tab includes:

1) Start/end date filters.

2) Limit control.

3) Next/Previous navigation.

4) Inline edit/delete actions.

This turns history from passive display into an active data quality management surface.

J. Core Feature 7: Dashboard Aggregation and Macro Progress Visualization

Dashboard feature converts logged events into daily actionable insight.

Endpoint: `GET /api/v1/dashboard/today`

Input:

1) Optional `day`, defaults to current date.

Output (`DashboardTodayResponse`):

1) Day.

2) Macro pairs for calories, protein, carbs, fat, fibre.

3) Meal count.

4) Streak days.

1. Aggregation Engine

Implemented in `services/dashboard.py`:

1) `recompute_daily_summary` aggregates meal/item rows for a day.

2) Summary row is inserted or updated.

3) Goal row is ensured if absent.

4) `get_streak_days` scans backward daily summaries until break.

5) `get_today_dashboard` returns normalized macro pair structure.

2. Feature Quality Considerations

1) Remaining value can be negative for overshoot, which is informative and preserved.

2) Dashboard always returns complete macro structure, reducing client branching.

3) Meal mutation hooks force summary recomputation, ensuring near-real-time consistency.

3. Frontend Rendering

Today tab renders five metric cards with consumed and remaining/target context. It also displays meal count and streak in caption format.

This design intentionally favors quick readability over visual density. The goal is immediate interpretation for daily decision-making.

K. Core Feature 8: Streak Calculation as Habit Reinforcement Signal

Streak is a small but important behavioral feature.

Logic:

1) Start from selected day.
2) Check if summary exists and meal_count > 0.
3) If yes, increment streak and move one day back.
4) Stop at first day without logs.

Why implemented this way:

1) Uses already materialized summary table.
2) Avoids expensive event-level scans each request.
3) Keeps mental model straightforward for users.

Streak is not gamification-heavy in current scope; it acts as a gentle continuity signal tied to actual logging behavior.

L. Core Feature 9: Contextual Nutrition Chat

Chat feature provides simple suggestion support based on current intake context.

Endpoint: `POST /api/v1/chat`

Input:

1) `message` required.
2) `context_day` optional.

Output:

1) `response`.
2) `context_day`.

1. Service Logic

In `create_chat_reply`:

1) Resolve context day.
2) Fetch dashboard for that day.
3) Build response using `_build_response`.
4) Persist user and assistant messages.
5) Commit and return response.

`_build_response` checks:

1) Protein gap.
2) Fibre gap.

3) Calorie overshoot.

4) Prompt keywords (for example "dinner" or "eat") for suggestion style.

This keeps responses grounded in user data and avoids purely generic outputs.

2. Chat Design Boundaries

The feature intentionally avoids medical claims and advanced diagnosis. It is framed as practical meal guidance. This boundary is important both for scope control and safe product positioning.

3. Frontend Integration

Chat tab provides:

1) Context day picker.

2) Scrollable chat history from session state.

3) Chat input with spinner feedback.

4) Clear chat action.

Backend-persisted chat messages provide durable records even though current UI session history is client-managed for immediate display.

M. Core Feature 10: Unified Error Handling and Client Robustness

Feature development included explicit error pathways across server and client.

Backend patterns:

1) Domain errors from services become `HTTPException` with appropriate status.

2) Missing resources return `404`.

3) Validation errors are automatically surfaced by FastAPI/Pydantic.

Frontend patterns:

1) `BackendClient` centralizes request logic.

2) Non-expected status codes are converted to `ApiError`.

3) Last error is stored in session state and displayed consistently.

4) Known `404` edit/delete cases trigger graceful refresh messaging.

Why this matters:

1) User trust depends on predictable failure behavior.

2) Debugging speed improves when errors are normalized.

3) Feature-level code remains cleaner because transport concerns are centralized.

N. Core Feature 11: Frontend Modularization for Maintainable Delivery

Although backend-focused, the frontend was developed as modular components to support clean feature mapping.

Module roles:

1) `app.py`: app shell, tab orchestration, top-level error display.
2) `client.py`: API abstraction and transport concerns.
3) `state.py`: session initialization and shared state helpers.
4) `ui_today.py`: log + dashboard + same-day meal edits.
5) `ui_history.py`: filters + pagination + historical edits.
6) `ui_chat.py`: conversational interface.
7) `ui_settings.py`: backend URL and user identity settings.

This decomposition mirrors backend domain split and accelerates iterative feature changes without creating monolithic UI logic.

O. Optimization Work Done During Feature Development

Feature development included practical optimization decisions aligned with project scale.

1) Deterministic parser-first strategy avoids expensive inference for common inputs.
2) Alias map built from seeded foods reduces repeated fuzzy operations.
3) Daily summaries materialized instead of full re-aggregation on every dashboard read.
4) Pagination in history prevents oversized payloads.
5) Reusable API client avoids duplicated network and error code.
6) Typed schemas reduce runtime ambiguity and rework.

These optimizations are targeted and low complexity, fitting the project's maintainability goals.

P. Data Integrity Behaviors Built into Features

Across features, several integrity patterns were implemented:

1) User scoping enforced in meal CRUD queries.
2) Cascade deletion of meal_items with meal delete.
3) Unique user-day summary row maintained through table constraint + update flow.
4) Dual-day recomputation on meal date edits prevents summary drift.
5) Goal auto-provisioning avoids null-target calculations.

These are not separate infrastructure features; they are embedded inside domain workflows to preserve consistency in normal use.

Q. Developer Experience Features

Feature development also included choices improving reproducibility and local iteration:

1) Startup auto-creation of tables.
2) Seed food auto-initialization on demand.
3) Health endpoint for quick runtime check.
4) Default local backend URL and demo user in frontend state.
5) Simple run commands in backend/frontend README files.

These choices reduce setup friction and make it easier to validate feature behavior repeatedly.

R. Feature Validation Approach

Formal automated test suites are not yet the dominant mechanism in the current project phase, so feature validation was driven through scenario-based manual verification using the Streamlit client and endpoint behavior checks.

Key validation scenarios:

1) Log a meal with explicit quantities and verify totals.
2) Log a meal with missing quantities and verify defaults + confidence behavior.
3) Log unknown food and verify fallback item generation.
4) Edit meal text and verify item replacement + summary update.
5) Edit meal date and verify old/new day recomputation.
6) Delete meal and verify summary decrement.
7) Filter history across date range and navigate pages.
8) Query dashboard for chosen day and verify macro pair consistency.
9) Ask chat guidance and verify context-sensitive response.
10) Trigger validation errors (empty text, invalid ranges) and verify clear messages.

This validation style ensures features are tested as users actually experience them.

S. Feature-Level Trade-Offs and Why They Were Chosen

During development, several design choices were made consciously:

1) CSV alias storage vs normalized alias table: a. Chosen CSV for speed of implementation. b. Acceptable at current scope; normalization can come later.
2) Full meal-item replacement on updates vs partial patching: a. Chosen replacement for deterministic behavior and simpler logic. b. Reduces mutation complexity and inconsistency risk.
3) Recompute summary on each mutation vs incremental delta updates: a. Chosen recompute for correctness clarity. b. Acceptable performance at current scale.

4) Token-based auth with persistent bearer sessions: a. Chosen to keep user isolation explicit across all protected endpoints. b. Supports production hardening through expiry/revocation and stricter claims policies.
5) Deterministic chat response builder vs LLM-heavy orchestration: a. Chosen for cost/predictability. b. Extensible to richer recommendation engines later.

These trade-offs show a consistent pattern: optimize for correctness, clarity, and shipping complete workflow value in the current phase.

T. Feature Development Outcomes

By the end of current development, NutriFlow delivers the complete MVP capability set required for a backend specialization project:

1) Natural-language meal logging with parser-backed normalization.
2) Persistent macro computation with item-level transparency.
3) Full meal lifecycle management (create, read, update, delete).
4) Day-level dashboard with macro targets, remaining values, and streaks.
5) Contextual nutrition chat grounded in user day summary.
6) Usable frontend surface that exercises all core backend features.

The product experience achieved by this feature set is coherent: users can move from meal entry to insight to action without leaving the system or performing complex manual steps.

U. Gaps and Planned Feature Enhancements

Feature development intentionally leaves room for next-phase expansion. The most relevant enhancements are:

1) Expanded food corpus and richer alias coverage for higher parse accuracy.
2) Optional LLM-assisted parser fallback for ambiguous text only.
3) Dedicated goal management APIs and UI controls.
4) Better confidence visualization and assumption explanation in frontend.
5) Weekly/monthly trend features and longitudinal insights.
6) Strong authentication and authorization layer.
7) Production-ready observability and rate limiting.
8) Optional caching layer for hot reference/summary reads.

Each enhancement can be added without major re-architecture because current feature modules already separate concerns cleanly.

V. End-to-End Feature Walkthrough Example

A representative flow demonstrates how developed features work together:

1) User opens Today tab and enters: "2 roti, dal, curd".
2) Frontend posts to meal creation endpoint with bearer token credentials.
3) Backend ensures user + goals, seeds foods if needed, parses text, stores meal/items, recomputes summary.
4) API responds with parsed items, totals, and confidence.
5) UI reruns and fetches dashboard for selected day.
6) User sees consumed/remaining macro cards and updated meal list.
7) User edits one meal entry in History tab for correction.
8) Backend reparses/replaces items and recomputes affected day summaries.
9) UI reflects corrected totals.
10) User asks chat: "What should I eat for dinner?"
11) Backend reads day gaps and responds with concise suggestion.
12) Chat messages are stored for traceability.

This sequence illustrates that feature development achieved functional completeness of the product loop, not just independent API availability.

W. Engineering Lessons from Feature Development

Several lessons emerged during implementation:

1) For behavior-driven products, correction features are as important as creation features.
2) Storing assumptions and confidence dramatically improves transparency in approximate systems.
3) Materialized daily summaries simplify both performance and product logic.
4) Thin endpoints + rich services improve maintainability and testability.
5) Simple deterministic logic often delivers better early-stage product reliability than complex AI-heavy pipelines.
6) Frontend integration should happen early; many backend edge cases emerge only in full-flow usage.

These lessons can guide the next iterations and also inform similar backend-centric product implementations.

X. Module-by-Module Implementation Notes

To provide additional implementation clarity, this subsection summarizes feature-level responsibility distribution across concrete files and why each separation was valuable during development.

1) `backend/app/services/meals.py`: This is the operational center of meal lifecycle features. It owns creation, update, delete, and history retrieval logic. Keeping these workflows together made it easier to enforce consistent summary recomputation behavior and shared helper usage (`_parsed_to_item`, `_items_totals`). It also

reduced duplication between create and update paths, especially around parser integration and totals conversion.

1) `backend/app/services/parser.py`: Parser concerns were intentionally isolated from persistence concerns. This made it possible to reason about parsing outcomes as pure data (`ParsedItem`, `ParsedMeal`) before any database write. During iteration, this separation accelerated parser refinements because changes did not require query-level rewrites.
1) `backend/app/services/dashboard.py`: Aggregation and streak logic were centralized in one module so all features consuming daily metrics shared one source of truth. This prevented mismatched computations between dashboard endpoint and chat context generation. It also ensured meal mutation handlers could call one stable recomputation function.
1) `backend/app/services/chat.py`: Chat response generation was isolated so contextual guidance could evolve independently of meal/dashboard contracts. This was useful when refining suggestion phrasing and gap interpretation rules. The module boundaries allow replacing rule-based generation with richer model-backed logic later without touching API endpoint signatures.
1) `backend/app/api/v1/endpoints/*.py`: Endpoint modules remain intentionally thin. Their job is dependency resolution, request-to-service delegation, and HTTP error mapping. This pattern improved readability during debugging, because transport errors and domain errors remained clearly separated.
1) `frontend/client.py`: A dedicated client abstraction removed repetitive request code across tabs and normalized error processing. This was important during feature growth because all tabs inherited consistent behavior for timeouts, status handling, and payload serialization.
1) `frontend/ui_today.py`, `frontend/ui_history.py`, `frontend/ui_chat.py`: Splitting UI by feature domain allowed faster iteration cycles. For example, history pagination improvements did not risk regressions in chat rendering, and dashboard metric card changes did not affect history filtering logic.
1) `backend/app/schemas/*.py`: Typed schemas played a major role in feature stability. Field constraints caught invalid inputs early, and explicit response models reduced accidental contract drift while evolving service code.

This module-level decomposition is one reason the project could expand feature depth without architectural rewrites. Each feature remained locally understandable while still participating in shared workflows such as user bootstrap, summary consistency, and standardized error propagation.

Y. Conclusion of Feature Development

Feature development in NutriFlow successfully transformed a scoped set of business requirements into a working, integrated application where users can log meals naturally, understand day-level nutritional status, correct historical records, and receive context-aware guidance. The implementation emphasizes practical usability and deterministic consistency while preserving extension points for future intelligence and scale. By building features as complete

vertical slices and grounding them in explicit service/database contracts, the project achieved a coherent and defensible architecture-to-product alignment suitable for both academic reporting and real-world evolution.

VI. Technologies Used

NutriFlow is built with a practical, backend-first stack chosen for fast implementation, maintainability, and production migration readiness.

A. Backend and API Layer

1) FastAPI (Python): FastAPI is the core web framework used to expose versioned REST endpoints for authentication, meals, dashboard, and chat. It provides high development speed, automatic OpenAPI documentation, and strong request/response validation through typed schemas. Real-life usage: Startups and internal platforms often use FastAPI to ship data-driven APIs quickly while preserving code quality and clear contracts. Example applications: recommendation APIs, analytics backends, IoT control APIs, internal enterprise microservices.
1) Uvicorn (ASGI server): Uvicorn runs the FastAPI app in local and production environments with efficient asynchronous request handling. Real-life usage: commonly used as the runtime server for modern Python API deployments behind Nginx or load balancers. Example applications: high-concurrency webhook handlers and API gateways.

B. Data and Persistence Layer

1) SQLAlchemy 2.x (ORM): SQLAlchemy is used for relational modeling, entity relationships, and query composition. It allows the codebase to remain portable across SQLite (development) and PostgreSQL/MySQL-compatible relational systems in production. Real-life usage: teams use SQLAlchemy to enforce clean data models, FK relationships, and transaction-safe operations in business-critical systems. Example applications: fintech ledgers, healthcare record backends, SaaS tenant databases.
1) SQLite (development) and PostgreSQL/RDS-ready design (production): SQLite keeps local setup lightweight for rapid iteration, while the schema and service patterns are designed to migrate cleanly to production-grade managed databases. Real-life usage: local prototyping with SQLite followed by cloud-hosted relational DB adoption for scale and concurrency. Example applications: MVP products moving to managed cloud databases after initial validation.

C. Validation, Configuration, and Security

1) Pydantic v2: Pydantic enforces payload validation and typed response contracts at the API boundary. Real-life usage: strict input/output validation reduces production bugs and integration mismatches across teams. Example applications: public APIs requiring stable contracts for multiple client applications.

1) python-dotenv and environment-based config: runtime values (DB URL, token settings, provider keys) are loaded from environment variables to separate code from secrets. Real-life usage: secure configuration management across dev, staging, and production. Example applications: any cloud-deployed app requiring environment-specific behavior.
1) Token-based authentication with hashed storage: bearer tokens are issued to users and persisted as hashes with expiry/revocation checks. Real-life usage: session management for SPA/mobile + API systems where stateless access control and revocation are required. Example applications: account-based productivity platforms, subscription dashboards, internal admin tools.

D. Frontend and Experience Layer

1) Streamlit: Streamlit is used for rapid delivery of a functional web UI with tabs for Today, History, Chat, and Settings. Real-life usage: teams use Streamlit for operational tools, analytics products, and early-stage customer-facing prototypes. Example applications: BI dashboards, model monitoring consoles, health/fitness tracker interfaces.
1) Requests (HTTP client): The frontend uses a centralized API client for backend communication and unified error handling. Real-life usage: client abstractions simplify authentication headers, retry/error behavior, and maintainability. Example applications: web and scripting clients interacting with internal or third-party APIs.

E. AI Integration Layer

1) Multi-provider LLM connector pattern (Groq, Gemini, Mistral, Mock fallback): NutriFlow integrates LLMs for chat and parser-assist flows through a provider-factory abstraction, with deterministic fallback when external calls fail. Real-life usage: production systems often support multiple model providers to reduce vendor risk and improve resiliency. Example applications: support copilots, context-aware assistants, data-enrichment pipelines.

F. Cloud and Operations (Target Production Stack)

1) AWS deployment components (VPC, EC2/Elastic Beanstalk, RDS, Security Groups, cache): the deployment design follows cloud best practices with network isolation, managed database, and optional caching for hot paths. Real-life usage: small teams and enterprises deploy API-based products on AWS using this exact pattern for reliability and operational control. Example applications: SaaS APIs, e-commerce services, EdTech platforms, health-tracking apps.

G. Technology Selection Rationale

The stack was selected using three criteria:

1) Speed of delivery for an end-to-end backend specialization project.

2) Strong maintainability through typed contracts and modular service boundaries.
3) Straightforward migration path from local development to managed cloud production.

VII. Deployment

A. Deployment Flow (AWS)

NutriFlow can be deployed on AWS using a network-isolated, managed-infrastructure approach. The flow below describes how requests and data move through the deployment architecture.

1) Client access: Users access the Streamlit frontend over HTTPS. The frontend communicates with the FastAPI backend using authenticated API calls.
1) VPC setup: All backend infrastructure is placed inside a dedicated AWS VPC with public and private subnets. Public subnet: load balancer/reverse proxy entry point. Private subnet: application runtime and database tiers.
1) Security Groups: Security groups enforce least-privilege traffic rules. Frontend/ALB SG allows inbound HTTPS (443) from the internet. App SG allows inbound backend traffic only from frontend/ALB SG. DB SG allows inbound database port only from app SG. Cache SG allows inbound cache port only from app SG.
1) Application runtime (EC2 or Elastic Beanstalk): Option A - EC2: deploy FastAPI + Uvicorn as a systemd service on EC2 instances behind an ALB. Option B - Elastic Beanstalk (managed infra): deploy the same backend as an application environment with managed scaling, health checks, and rolling updates. Both options run the same application code and environment-variable configuration model.
1) Database layer (RDS): Move from local SQLite to Amazon RDS (PostgreSQL recommended for this architecture). RDS runs in private subnets with automated backups, patching windows, and Multi-AZ (if required).
1) Cache layer: Add Amazon ElastiCache (Redis) for hot-read acceleration and response optimization. Typical cache targets: food alias map, frequently requested dashboard reads, and short-lived chat context fragments.
1) Configuration and secrets: Runtime configuration is injected via environment variables; secrets should be managed through AWS Secrets Manager or SSM Parameter Store.
1) Observability and operations: Application and infrastructure logs stream to CloudWatch. Health checks are exposed via `/api/v1/health`. Alerts can be configured on error rate, latency, instance health, and DB utilization.
1) CI/CD and release path: Source push triggers build/test pipeline. Successful builds deploy to staging, then production with controlled rollout. If Elastic Beanstalk is used, deployment versions are managed per environment with rollback support.

B. Deployment Architecture Summary

In production, the recommended topology is: `User -> HTTPS -> Frontend -> ALB/Reverse Proxy -> FastAPI App (EC2 or Elastic Beanstalk) -> RDS (+ optional ElastiCache)`.

This design preserves the current code architecture while improving security, scalability, and operational reliability.

VIII. Conclusion

NutriFlow demonstrates that a focused, backend-first architecture can solve a high-drop-off product problem with practical engineering trade-offs. The implemented system successfully delivers the full daily loop of natural-language meal logging, structured macro computation, dashboard feedback, history-based correction, and context-aware chat guidance. The project validates that consistency-oriented design can create meaningful user value even when nutritional estimation is approximate.

A. Key Takeaways

1) Problem-first scope improves execution quality: limiting scope to the core behavior loop (log -> understand -> act -> correct) produced a complete and reliable implementation instead of a partially built feature set.
2) Modular monolith architecture is effective for this stage: separating API, services, schemas, and models provided maintainability and clear requirement-to-code traceability without distributed-system overhead.
3) Deterministic parsing with transparent assumptions is practical: storing confidence and assumptions alongside parsed items improves trust and supports correction workflows.
4) Materialized daily summaries simplify product logic: recomputing and storing day-level aggregates keeps dashboard and chat context consistent after meal mutations.
5) Reliability is strengthened by correction paths: update/delete support and history retrieval are essential in approximate systems where user edits are expected.
6) Resilient AI integration matters: provider abstraction with fallback behavior prevents external model failures from breaking core user workflows.

B. Practical Applications

The architecture and technology choices used in NutriFlow have direct real-world relevance across domains:

1) Health and fitness platforms: fast natural-language capture with interpretable feedback can improve adherence in calorie and macro tracking products.
2) Digital coaching tools: context-aware assistance based on recent user state can support recommendations in nutrition, wellness, and habit-formation apps.
3) EdTech and productivity systems: the same pattern of event logging + daily aggregation + assistant guidance can

be reused for learning progress and task management.

4) Enterprise operational tools: modular API/service/data layering supports rapid internal tool development while keeping migration paths open to managed infrastructure.

These applications show that the project is not only academically complete, but also transferable to production-oriented software scenarios.

C. Limitations, Cost Implications, and Improvement Suggestions

Current limitations:

1) Nutritional precision is approximate, especially for mixed dishes and regional food diversity.
2) Food catalog and alias coverage are finite, affecting parse quality for long-tail inputs.
3) Chat recommendations are intentionally lightweight and not medical-grade advice.
4) Automated test coverage and production observability are still limited for large-scale operation.

Cost implications:

1) LLM usage introduces variable per-request cost and latency, especially under sustained chat/parser-assist traffic.
2) Managed cloud components (RDS, ElastiCache, load balancing, monitoring) increase monthly operating cost compared to local/single-node setups.
3) Scaling for concurrency (multi-instance app, managed DB tiers) requires ongoing spend and capacity planning.

Suggestions for improvement:

1) Expand curated food corpus and alias normalization to reduce unknown-food fallbacks.
2) Apply selective LLM invocation only for ambiguous inputs, with caching for repeated lookups, to control cost.
3) Add migration tooling, automated tests, and CI gates for regression safety.
4) Introduce richer observability (metrics, traces, SLO-based alerts) for production readiness.
5) Add stronger auth hardening, rate limiting, and secret-management controls for internet-facing deployments.
6) Extend analytics with weekly/monthly trends and goal-adjustment workflows for deeper user retention.

Overall, NutriFlow achieves its primary objective: transforming nutrition tracking from a high-friction manual task into a repeatable, insight-driven daily workflow. The implementation is technically coherent, operationally extensible, and well-positioned for next-phase improvements in precision, personalization, and scale.

IX. References

1) FastAPI. *FastAPI Documentation*. Available at: https://fastapi.tiangolo.com/.
2) FastAPI. *Deployment - FastAPI*. Available at: https://fastapi.tiangolo.com/deployment/.
3) Streamlit. *Streamlit Documentation*. Available at: https://docs.streamlit.io/.
4) Streamlit. *Basic concepts of Streamlit*. Available at: https://docs.streamlit.io/get-started/fundamentals/main-

concepts.

5) SQLAlchemy authors. *SQLAlchemy Documentation*. Available at: https://docs.sqlalchemy.org/.
6) Pydantic Services Inc. *Pydantic Documentation*. Available at: https://docs.pydantic.dev/latest/.
7) Alembic authors. *Alembic Documentation*. Available at: https://alembic.sqlalchemy.org/.
8) SQLite. *SQLite Documentation*. Available at: https://sqlite.org/docs.html.
9) PostgreSQL Global Development Group. *PostgreSQL Documentation*. Available at: https://www.postgresql.org/docs/.
10) Fowler, M. *Monolith First*. Available at: https://martinfowler.com/bliki/MonolithFirst.html.
11) Jones, M., Bradley, J. and Sakimura, N. *RFC 7519: JSON Web Token (JWT)*. Internet Engineering Task Force, 2015. Available at: https://datatracker.ietf.org/doc/html/rfc7519.
12) Amazon Web Services. *What is Amazon EC2?* Available at: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html
13) Amazon Web Services. *What is Amazon VPC?* Available at: https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html.
14) Amazon Web Services. *Control traffic to your AWS resources using security groups*. Available at: https://docs.aws.amazon.com/vpc/la security-groups.html.
15) Amazon Web Services. *What is Amazon Relational Database Service (Amazon RDS)?* Available at: https://docs.aws.amazon.com/Am
16) Amazon Web Services. *What is Amazon ElastiCache?* Available at: https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/Wha
17) Amazon Web Services. *What is AWS Elastic Beanstalk?* Available at: https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/Welcon
18) U.S. Department of Agriculture. *FoodData Central*. Available at: https://fdc.nal.usda.gov/.
19) Burke, L. E., Wang, J. and Sevick, M. A. *Self-Monitoring in Weight Loss: A Systematic Review of the Literature*. *Journal of the American Dietetic Association*, 2011. Available at: https://pubmed.ncbi.nlm.nih.gov/21185970/.
20) Peterson, N. D., Middleton, K. R., Nackers, L. M., Medina, K. E., Milsom, V. A. and Perri, M. G. *Dietary Self-Monitoring and Long-Term Success with Weight Management*. *Obesity*, 2014. Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC4149603/.
21) Raber, M., Liao, Y., Rara, A., Schembre, S. M., Krause, K. J., Strong, L., Daniel-MacDougall, C. and Basen-Engquist, K. *A Systematic Review of the Use of Dietary Self-Monitoring in Behavioral Weight-Loss Interventions: Delivery, Intensity and Effectiveness*. *Public Health Nutrition*, 2021. Available at: https://pubmed.ncbi.nlm.nih.gov/34412727/.